

TANGO: A Robust Qubit Mapping Algorithm via Two-Stage Search and Bidirectional Look

Kang Xu^{1,2}, Yukun Wang^{1,2,*}, Dandan Li³

¹Beijing Key Laboratory of Petroleum Data Mining, China University of Petroleum, Beijing 102249, China

²Key Lab of Processors, Institute of Computing Technology, CAS, Beijing 100190, China

³School of Computer Science (National Pilot Software Engineering School), Beijing University of Posts and Telecommunications, Beijing 100876, China

*wykun06@gmail.com (<mailto:wykun06@gmail.com>)

Abstract

Current quantum devices typically lack full qubit connectivity, making it difficult to directly execute logical circuits on quantum devices. This limitation necessitates quantum circuit mapping algorithms to insert SWAP gates, dynamically remapping logical qubits to physical qubits and transforming logical circuits into physical circuits that comply with device connectivity constraints. However, the insertion of SWAP gates increases both the gate count and circuit depth, ultimately reducing the fidelity of quantum algorithms. To achieve a balanced optimization of these two objectives, we propose the TANGO algorithm. By incorporating a layer-weight allocation strategy, the algorithm first formulates an evaluation function that balances the impact of qubit mapping on both mapped and unmapped nodes, thereby enhancing the quality of the initial mapping. Next, we design an innovative two-stage routing algorithm that prioritizes the number of executable gates as the primary evaluation metric while also considering quantum gate distance, circuit depth, and a novel bidirectional-look SWAP strategy, which optimizes SWAP gate selection in conjunction with preceding gates, improving the effectiveness of the mapping algorithm. Finally, by integrating advanced quantum gate optimization techniques, the algorithm's overall performance is further enhanced.

Experimental results demonstrate that, compared to state-of-the-art methods, the proposed algorithm achieves multi-objective co-optimization of gate count and circuit depth across various benchmarks and quantum devices, exhibiting significant performance advantages.

Index Terms: : quantum computing, qubit mapping, collaborative optimization.

I INTRODUCTION

Quantum computing is one of the most significant approaches in the post-Moore era. It has been widely applied to problems such as integer factorization, database searching, and solving linear systems of equations, demonstrating computational advantages over classical computers in specific tasks and achieving remarkable speedups. Currently, mainstream quantum computing solutions are provided by companies such as IBM[1], Google [2], and Intel [3], featuring qubit counts of 127, 54, and 49, respectively. In 2024, Google announced the launch of its high-performance quantum processing unit (QPU), Willow [4]. Compared to its predecessors, this processor achieves significantly reduced error rates, further paving the way for practical quantum computing.

Currently, the dominant model in quantum computing is the quantum circuit model, where each quantum operation is represented as a quantum gate. A typical quantum circuit consists of single-qubit gates and two-qubit gates, with the latter being executable only between directly connected qubits. In Noisy Intermediate-Scale Quantum (NISQ) devices, the connectivity between qubits is often restricted. In order to enable a given quantum circuit to execute on a connectivity-constrained quantum architecture, the original ideal quantum circuit must be transformed. This is typically achieved by inserting SWAP gates, which swap qubits to make non-adjacent qubits physically adjacent, allowing the corresponding quantum gates to be performed. This process, referred to as the qubit mapping problem, primarily aims to minimize the number of SWAP gates introduced during the mapping. This is crucial because the insertion of SWAP gates increases both the gate count and circuit depth, leading to longer runtime and additional noise, which ultimately diminishes the success rate of quantum algorithm execution.

To address this issue, numerous effective qubit mapping algorithms have emerged over the past few decades. Zulehner et al. [5] divided the circuit into layers and

employed an A* search to find optimal SWAP gates, though its time complexity grows exponentially with search depth. To mitigate this, Li et al. [6] introduced SABRE, a bidirectional traversal algorithm with a decay-based heuristic to reduce circuit depth and time complexity. Niu et al. [7] enhanced SABRE with simulated annealing to optimize initial mapping while considering gate noise, which effectively reduced additional CNOT gates and improved circuit fidelity. Furthermore, Zhang et al. [8] augmented the A* algorithm with node expansion and pruning operations, and designed the depth-focused heuristics to minimize circuit depth. Chang et al. [9] proposed a novel hexagonal architecture-based mapping algorithm, but its weight assignment only considers individual gate counts, neglecting prioritization between mapped and unmapped nodes, potentially affecting initial mapping quality.

Besides, Liu et al. [10] proposed NASSC, an optimization-aware routing method that reduces redundant gates via logic synthesis and gate cancellation, outperforming SABRE in both circuit depth and gate count. On the other hand, Li et al. [11] proposed SQGM to optimize the insertion of SWAP gates by storing single-qubit gates, achieving better circuit depth than NASSC when integrated with SABRE but lagging in CNOT gate count. McKinney et al. [12] introduced the MIRAGE framework, which reduces SWAP gates by leveraging mirror gates and optimizing their decomposition into the iSWAP family, offering new insights for future quantum circuit mapping. Tan et al. [13] proposed a hybrid strategy for Dynamically Field-Programmable Qubit Arrays (DPQA), combining greedy and optimal methods based on gate count. Their work shows that DPQA-based circuits have lower expansion overheads than fixed-grid architectures, advancing programmable quantum circuits for neutral atom quantum computers. In conclusion, The primary optimization goals of these algorithms typically include minimizing the number of inserted SWAP gates [14, 15, 16, 17, 18, 19, 20, 21, 22, 23], reducing quantum circuit depth [24, 25, 26, 27], and lowering the overall error rate of the circuit [28, 29, 30, 31, 32, 33]. However, existing methods rarely balance the collaborative optimization of quantum gate count and circuit depth, and they lack the capability for further re-optimization of quantum circuits, thus presenting a significant direction for future research.

In this paper, we propose TANGO, a method for jointly optimizing quantum circuit gate count and depth. First, a discriminative function is formulated, incorporating a parameter that balances the impact of qubit placement on both mapped and un-

mapped nodes, thereby optimizing the initial mapping selection. Then, we propose an innovative two-stage routing algorithm, with the number of executable gates as the primary evaluation metric. Quantum gate distance, circuit depth, and a novel bidirectional look SWAP strategy are subsequently considered as auxiliary evaluation metrics, effectively identifying SWAP gates that achieve optimal exchange efficiency and minimal circuit depth, while also enhancing the potential for subsequent gate cancellation optimization. Finally, by integrating advanced quantum gate optimization techniques, the total gate count and circuit depth are further reduced. Experimental results demonstrate that the algorithm achieves multi-objective co-optimization of gate count and circuit depth across multiple large-scale datasets, further confirming its efficiency and broad applicability.

The structure of this paper is as follows: Sec. II introduces the basic concepts and background of the circuit mapping problem; In Sec. III, the co-optimization mapping algorithms are presented, including the proposed initial mapping method and the innovative two-stage routing algorithm; Sec. IV evaluates the algorithm's performance and analyzes the experimental results; The key findings and contributions of this work are concluded in Sec. V.

II PRELIMINARIES

II-A Quantum Computing and Quantum Gate

Quantum computing utilizes qubits, which differ from classical bits that are either 0 or 1. A qubit can be in states $|0\rangle$ or $|1\rangle$, or a superposition of both, expressed as $|\phi\rangle = \alpha|0\rangle + \beta|1\rangle$. Here, α and β are complex numbers, and they satisfy the normalization condition $|\alpha|^2 + |\beta|^2 = 1$.

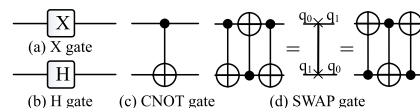


Figure 1: Basic quantum gates in quantum computing

Quantum gates are the fundamental operations used to manipulate quantum states. Single-qubit gates act on an individual qubit. For example, as shown in Fig. 1, the X gate flips the states: $X|0\rangle = |1\rangle$ and $X|1\rangle = |0\rangle$. The Hadamard(H) gate generates states of equal superposition: $H|0\rangle = (|0\rangle + |1\rangle) / \sqrt{2}$ and $H|1\rangle = (|0\rangle - |1\rangle) / \sqrt{2}$. Two-qubit gates, on the other hand, act on a pair of qubits simultaneously. The most common example is the CNOT (CX) gate, which applies an X gate to the target qubit if the control is $|1\rangle$ state. The SWAP gate exchanges the states of two qubits and can be decomposed into three CNOT gates. In addition to these, there are various higher-dimensional quantum gates, which can ultimately be decomposed into a linear combination of single-qubit and two-qubit gates.

II-B Quantum Device and Quantum Circuit

The quantum device is represented as $AG(Q_p, E)$, where Q_p are physical qubits, and E are connections between them. In NISQ-era quantum architectures (see Fig. 2), each node represents a physical qubit. It is evident that two-qubit gates can only be performed between physically connected qubits, while single-qubit gates don't. Among these architectures, the IBM Q20 architecture, with its dense connectivity, is often selected to validate the performance of mapping algorithms.

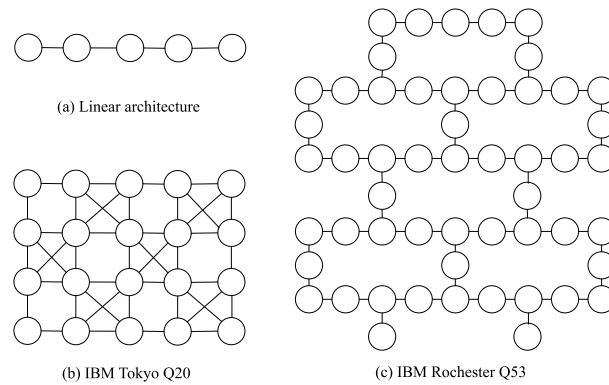


Figure 2: The coupling graphs of real quantum devices

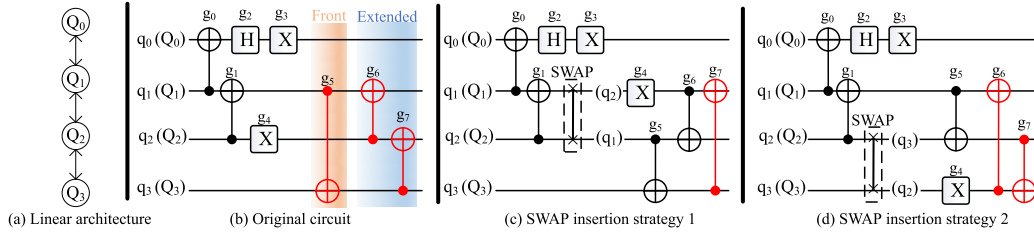


Figure 3: The basic initial mapping relationship and the two given strategies for inserting SWAP gates

The quantum circuit is typically represented as $LC(q, g)$, where q_i denotes the logical qubits and g represents the gates. Each line in the circuit diagram represents a qubit, with quantum gates ordered left to right by execution. For example, in Fig. 3(b), the quantum circuit consists of 4 qubits and 8 quantum gates, with single-qubit gates like X and H , and two-qubit gates like g_0 and g_5 , which are represented as $CX(q_1, q_0)$ and $CX(q_1, q_3)$, respectively. Finally, after all gates are executed, the result of the circuit is obtained through measurements at the end of the quantum circuit.

II-C Qubit Mapping and Routing Problem

A quantum circuit $LC(q, g)$ and quantum device architecture $AG(Q_p, E)$ are considered, restricted to single-qubit and CNOT gates for simplicity.

The qubit mapping problem involves establishing an initial one-to-one correspondence between logical qubits and physical qubits, denoted as $\pi : \{q_l \rightarrow Q_p\}$. This initial mapping can be either randomly assigned or determined by a simple heuristic. As illustrated in Fig. 3(b), the mapping is represented by $\pi^{init} : \{q_i \rightarrow Q_i\}$, which satisfies the architectural constraints shown in Fig. 3(a) for certain CNOT gates (e.g., g_0 and g_1) within the circuit.

The routing problem is defined as follows: Given an initial mapping, certain two-qubit gates in the circuit may involve logical qubits mapped to physically distant qubits. To address this, SWAP gates are dynamically inserted to adjust the mapping, ensuring that the logical qubits in a two-qubit gate are mapped to directly

connected physical qubits, enabling gate execution. As shown in Fig. 3(b), the gates marked in red (g_5, g_6, g_7) cannot be executed under the mapping π^{init} and the gate dependency constraints, two feasible SWAP insertion strategies are illustrated in Fig. 3(c) or Fig. 3(d).

II-D Circuit Depth and Single Qubit Gate Storage

To effectively track and optimize the circuit depth for each qubit in a quantum circuit, the DP is introduced. Specifically, $DP(Q)$ represents the current depth of a physical qubit Q , initialized to 0. For single-qubit gates, the DP value of the corresponding node is incremented by 1. For two-qubit gates acting on q_0 and q_1 , the depths of Q_{q_0} and Q_{q_1} are updated as $DP(Q_{q_0}) = DP(Q_{q_1}) = \max(DP(Q_{q_0}), DP(Q_{q_1})) + 1$. For SWAP gates, the update rule is $DP(Q_{q_1}) = DP(Q_{q_0}) = \max(DP(Q_{q_1}), DP(Q_{q_0})) + 3$, reflecting their decomposition into three CNOT gates.

In addition, the execution order of single-qubit gates impacts circuit depth, making their storage in SG essential. During circuit execution, all encountered single-qubit gates are stored in SG . When a two-qubit gate (q_0, q_1) satisfying device connectivity constraints is met, both the single-qubit gates stored in $SG(Q_{q_0})$ and $SG(Q_{q_1})$ will be popped from SG and applied to the circuit. For a two-qubit gate that does not satisfy the architecture constraints, SWAP insertion is preceded by computing $s = \arg \min(DP(Q_{q_0}), DP(Q_{q_1}))$ and removing $k = \min(|DP(Q_{q_0}) - DP(Q_{q_1})|, \text{len}(SG(s)))$ single-qubit gates from $SG(s)$ to balance DP values.

III OUR METHOD

This section primarily presents the proposed TANGO method. First, we introduce the dual-factor optimization for initial mapping, which improves mapping quality by balancing the impact of qubit placement on mapped and unmapped nodes. Next, a novel two-stage qubit routing optimization method is proposed, which considers factors such as the number of executable gates, circuit depth, and a bidirectional SWAP look strategy, thereby enabling the co-optimization of gate count and circuit depth.

III-A Dual-Factor Optimization for Initial Mapping

The objective of the initial mapping is to determine an optimal placement of qubits on a connectivity-constrained physical architecture, minimizing SWAP gates inserted during the routing process, thereby reducing the overall gate count and circuit depth. To achieve this, we propose a dual-factor evaluation function to enhance the initial mapping process. First, gate weights within the circuit should be defined, as they determine the mapping priority of the qubits.

Naturally, for a quantum circuit $LC(q, g)$, gate weights will gradually decrease due to gate dependency, as it directly impacts the execution of subsequent gates. While some existing methods [20, 9] assign a unique and monotonically decreasing weight to each gate, they fail to adequately account for the weight distribution of parallel gates within the circuit. This insufficient consideration of weight information may adversely impact the effectiveness of the initial mapping. Therefore, we define the weights based on the parallelism rules of gates in the circuit. Inspired by reward function design in reinforcement learning, a decremental form is employed to define the weight $W_g(g_i)$ for each gate, specifically represented as:

$$W_g(g_i) = \gamma^{layer_i}, \quad (1)$$

where γ indicates the initial weight, g_i represents the i -th gate in the circuit, and $layer_i$ denotes the layer of the quantum circuit where g_i is located.

Once the weight for each quantum gate is defined, the priority of each qubit is determined by the sum of the weights of the gates involving the current qubit. The weight for each qubit is defined as:

$$W_q(q_i) = \sum_{g_j} W_g(g_j), \quad \text{where } q_i \in g_j. \quad (2)$$

Similarly, for the weight definition between two qubits, it is given by the sum of the gate weights involving both qubits, expressed as:

$$e_{qw} = \sum_i W_g(g_i), \quad \text{where } q \text{ and } w \in g_i. \quad (3)$$

After calculating the weights for all qubits, the qubits with higher weights are prioritized for execution, as their associated gates are more concentrated at the front of the circuit. Specifically, by sorting the qubits based on $W_q(q_i)$, the qubit with the

highest weight is mapped first, followed by qubits with progressively lower weights. The qubit with the highest weights is preferentially mapped to the center point of the quantum device, as this is considered the core position within the architecture. For the remaining logical qubits to be mapped, the algorithm primarily considers two key factors: first, the impact of placing a qubit at a designated location on the already mapped qubits; second, the effect on the unmapped qubits. By considering both factors, the overall impact of qubit placement can be comprehensively evaluated.

To analyze the first factor, we design a scoring function to assess the impact on mapped nodes:

$$f_p(q, Q_v) = \sum_w \left(1 - \frac{\text{dis}(Q_v, Q_w)}{\text{dia}(AG)} \right) * e_{qw}, \quad (4)$$

where q is the qubit node to be mapped to the quantum device, Q_v represents the corresponding position on the quantum device, w is a previously placed logical neighbor of q , $\text{dia}(AG)$ is the diameter of the architecture AG , and $\text{dis}(Q_v, Q_w)$ denotes the distance between these two nodes on the quantum device. The motivation behind this function is to place q as close as possible to its logical neighbor w and to ensure that the weight between the two nodes is larger, thereby maximizing the value of the objective function f_p and potentially increasing the likelihood of executing subsequent gates.

The second factor involves considering the unplaced logical neighbors of qubit q , for which we define the function f_{up} as follows:

$$f_{up}(q, Q_v) = \frac{(f(Q_v) - |q'|) * \sum_u e_{qu}}{\max(f(Q_v), |q'|)}, \quad (5)$$

where u represents an unplaced logical neighbor of q , $|q'|$ denotes the number of unmapped logical qubits connected to q , and $f(Q_v)$ represents the number of available neighboring nodes at position Q_v on the quantum device. Similarly, the motivation behind this function is to identify suitable free positions for placing q 's logical neighbors, prioritizing qubits with higher weights. If the available positions around Q_v are insufficient to accommodate all of q 's neighbors, which results in the function value being negative, effectively acting as a penalty term.

During the calculation of both scoring functions, the values of f_p and f_{up} may vary significantly. To address this, both functions are standardized using Z -scores to bring them to the same scale, resulting in f_{p_z} and f_{up_z} . Furthermore, adjustable parameters α and β are introduced to appropriately weight these functions, thereby achieving a balanced relationship between them. The final scoring function is defined as:

$$F(q) = \arg \max_{Q_v} \left(\alpha * f_{p_z}(q, Q_v) + \beta * f_{up_z}(q, Q_v) \right). \quad (6)$$

Based on this approach, the placement position for each qubit q in the quantum architecture is determined by selecting the position Q_v that maximizes the function value F . This process is iteratively applied to all qubits in the logical quantum circuit until all logical qubits are successfully mapped to physical nodes in the quantum device. Ultimately, a mapping relationship π^{init} is established to guide subsequent qubit routing.

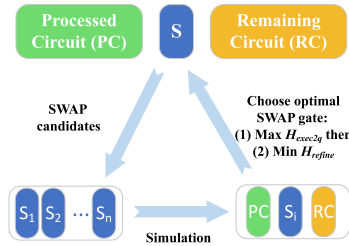


Figure 4: The workflow of the two-stage routing algorithm for searching SWAP gates

III-B Two-Stage Qubit Routing Optimization Algorithm

After obtaining the initial mapping, additional SWAP gates are often needed to modify the qubit mapping, ensuring the execution of two-qubit gates that may not satisfy the connectivity constraint. To optimize both the number of additional gates and the circuit depth introduced during the mapping process, we propose an innovative two-stage routing algorithm. This algorithm uses the executable gate count as the backbone evaluation metric, with circuit-related features serving as auxil-

iary decision factors to effectively identify efficient SWAP gates. The workflow is illustrated in Fig. 4.

Initially, the SWAP candidates is defined as $\text{SWAP}_{\text{candidate}} = \{\text{SWAP}_i\}$, where each SWAP_i involves at least one qubit that appears in the first k two-qubit gates of the remaining circuit. This strategy ensures that the size of the SWAP candidate set never exceeds $|EG|$, the original search space size determined by the total number of edges in the architecture, thereby effectively reducing the computational resource requirements and enhancing the algorithm's efficiency.

In the first stage, we define H_{exec2q} as a scoring function to evaluate each SWAP gate within the $\text{SWAP}_{\text{candidate}}$. Most existing heuristic methods for SWAP gate selection typically rely on the distance between subsequent gates after the mapping transformation. However, we find that the strategy based on the number of executable gates directly addresses the mapping problem more effectively, as opposed to indirectly assessing gate distances to determine the insertion of SWAP gates. This H_{exec2q} function primarily focuses on maximizing the number of two-qubit gates that can be executed by a single SWAP gate, thereby ensuring that the current SWAP gate is optimal within the local search space. Consequently, the function H_{exec2q} can be defined as follows:

$$H_{\text{exec2q}}(\text{SWAP}_i) = \text{Execute}(\text{Gates}_{2q}), \quad (7)$$

where $\text{Execute}(\text{Gates}_{2q})$ represents the number of two-qubit gates that can be executed in the quantum circuit after inserting the SWAP_i gate. A higher H_{exec2q} value indicates that the current SWAP gate yields a greater benefit within the local scope, as more two-qubit gates can be executed. By selecting SWAP_i from the SWAP candidate set and calculating its corresponding function based on Eq. 7, the maximum value MES is obtained as: $MES = \max(H_{\text{exec2q}})$. When $MES = 0$, it indicates that no SWAP gate can enable the execution of any two-qubit gates. If $MES \neq 0$, multiple SWAP gates may yield the same value. In such cases, further refinement is required to select the optimal SWAP gate by considering additional factors, such as gate distance and circuit depth, which constitutes the objective of the second stage.

For the scenario where $MES \neq 0$, motivated by the fact that the gate distance, circuit depth, and routing-based circuit optimization help guide the insertion of SWAP gates, thereby optimizing both the quantum circuit depth and the number of

additional gates. Based on this, we define the following refined scoring function H_{refine} , with the objective of minimizing its value:

$$H_{refine} = H_{decay} + DP_{max} - Reward, \quad (8)$$

where H_{decay} highlights the impact of SWAP insertion on the distance between the remaining gates in the circuit and is expressed as:

$$H_{decay} = \max(\text{decay}(Q_i), \text{decay}(Q_j)) * \left(\frac{1}{|F|} \text{Dis}(\pi, F) + w * \frac{1}{|E|} \text{Dis}(\pi, E) \right). \quad (9)$$

Here, the term “decay” refers to an increase in the decay value δ when a SWAP gate is executed between qubits Q_i and Q_j , where δ ranges from $[0, 1]$. This mechanism encourages the algorithm to prioritize qubits that have not recently participated in a SWAP gate, thereby effectively reducing circuit depth.

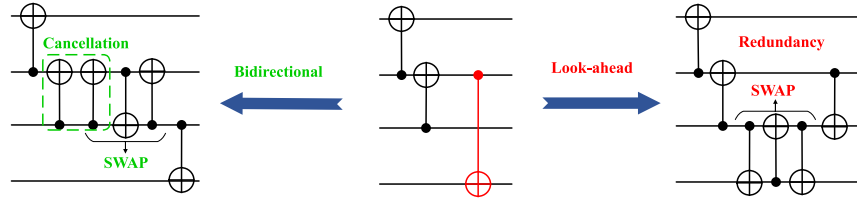


Figure 5: An example of the motivation for the bidirectional SWAP gate search

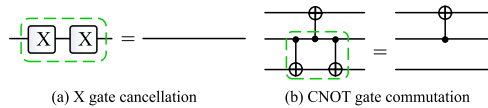


Figure 6: Rules for gate cancellation and commutation of X and CNOT gates

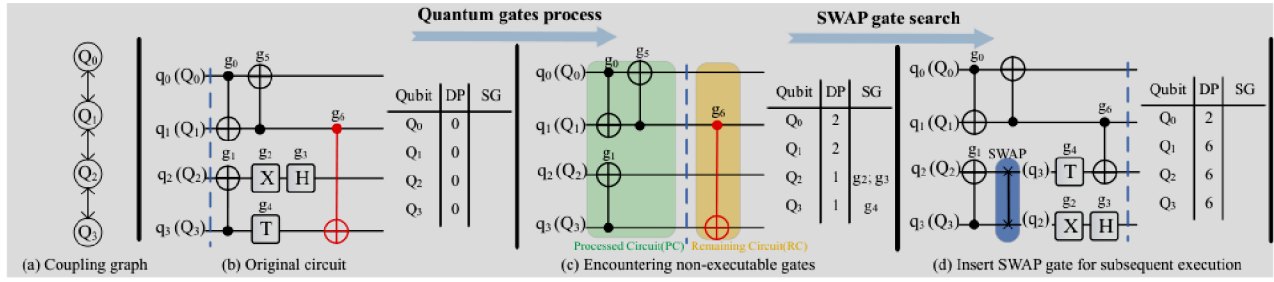


Figure 7: The example of two-stage qubit routing algorithms within a circuit

The impact of a SWAP gate on subsequent quantum gates is categorized into two types of gate distance contributions: F , representing the front layer, and E , representing the extended layer, are both shown in Fig. 3(b). The range of E is usually determined by a predefined look-ahead window, which specifies the number of gates considered within the extended layers. Besides, a discount factor is applied to the gates in the extended layer, as the gates in the front layer F need to be executed first to enable the execution of subsequent dependent operations.

As discussed in the background section, to optimize circuit depth during the mapping process, we introduce a variable DP to track the depth of each qubit. Specifically, each time a single-qubit gate is applied to a qubit, its corresponding DP value increases by 1, reflecting the qubit's current depth within the circuit. Additionally, we define a set SG to store the single-qubit gates for a given qubit. If the CNOT gates are executable, these single-qubit gates are applied to the relevant qubits accordingly.

The term DP_{max} , defined as $DP_{max} = \max(DP_i)$, indicates the maximum depth of the circuit after inserting a SWAP gate at the current position, aiming to reduce the insertion of SWAP gates into qubits with higher depths.

Moreover, the term *Reward* is introduced as a novel bidirectional SWAP marker, with its motivation illustrated in the Fig. 5. Unlike conventional look-ahead approaches that focus solely on the impact of SWAP insertion on remaining gates, we also consider its influence on preceding gates, forming a bidirectional look approach. This is because a SWAP operation may facilitate optimization with preceding gates, thereby reducing both the number of gates and the overall circuit depth.

Initially set to 0, *Reward* is updated based on a tracking set *REC*, which records the most recent gate executed for each qubit. If $REC[Q_i] = REC[Q_j]$, this implies that Q_i and Q_j share the same gate. In this case, a positive *Reward* is assigned, reflecting the potential for the SWAP gate to cancel out the two-qubit gates preceding it, further simplifying the quantum circuit. Finally, by minimizing the function H_{refine} , we determine the optimal SWAP gate for insertion, enabling the execution of subsequent gates.

For the second scenario, where $MES = 0$, the process becomes relatively straightforward. The shortest-distance CNOT gate from the front layer that cannot be executed is identified, and a SWAP gate is applied to reduce its distance by 1. Simultaneously, it is ensured that this SWAP gate satisfies the $\min(H_{decay})$ condition among the available candidates. This strategy enables the algorithm to efficiently execute high-priority gates while minimizing the distances of subsequent gates, hence optimizing the overall performance of the quantum circuit.

By repeatedly performing this routing process, all quantum gates in the circuit are eventually executed. Similarly to the SABRE algorithm's initial mapping generation phase, bidirectional traversal techniques are applied. For the entire unmapped quantum circuit, denoted as the Original Circuit (OC), the initial mapping and routing algorithm is applied to make the OC executable on the architecture. The OC is then reversed to obtain OC_{RS} , and the mapping relationship generated at the end of the previous mapped circuit is input into OC_{RS} . After completing the routing phase and executing all quantum gates, the circuit is reversed again to obtain the OC, and the routing process is performed again. Finally, we evaluate the minimum circuit depth achieved over these three runs. It is important to clarify that the focus of our approach's rerunning lies in the circuit depth or total gate count obtained during the bidirectional traversals, rather than merely for generating a good initial mapping. This insight helps prevent the significant computational resource waste that would occur within the SABRE algorithm.

To further enhance the performance of the mapped quantum circuit, we have integrated relevant circuit optimization strategies into our algorithm. As illustrated in Fig. 6, gate cancellation and commutation techniques are applied, which play a crucial role in significantly optimizing the quantum circuit. Specifically, for multiple single-qubit gates, such as two adjacent X gates, they can be directly canceled. Similarly, optimization rules can be applied to two-qubit gates, particularly CNOT

gates, where consecutive gates sharing the same control or target qubits can often be canceled or commuted. What's more, the adaptive decomposition of SWAP gates is determined based on the gate directions preserved in the tracking set REC , allowing the adaptive strategy to eliminate CNOT gates. It was found that the SWAP gate in the SQGM algorithm only has a single decomposition form and lacks this adaptive approach, which is critical for quantum circuit optimization. These optimizations effectively simplify the quantum circuit while maintaining functional equivalence, thus reducing the gate counts and the circuit depth.

To better understand our proposed two-stage routing optimization algorithm, we present an illustrative example of the overall routing process in 7. Fig. 7(b) presents the initial mapping relationship:

$$\pi^{init} = \{q_0 : Q_0, q_1 : Q_1, q_2 : Q_2, q_3 : Q_3\}. \quad (10)$$

Initially, DP is set to 0, and SG is initialized as null. During quantum circuit execution, single-qubit gates are stored in SG until an unexecutable CNOT gate, g_e , is encountered. The decision for SWAP insertion is based on the calculation results from Eq. 7. Specifically,

$$H_{exec2q}(SWAP(q_2, q_3)) = H_{exec2q}(SWAP(q_1, q_2)). \quad (11)$$

In the second stage, these two candidate SWAP gates are further evaluated to determine the optimal choice. It is observed that $DP_{max}(SWAP(q_1, q_2)) = 5$ is greater than $DP_{max}(SWAP(q_2, q_3)) = 4$. Besides, for $SWAP((q_2, q_3))$ gate, since $REC[Q_2] = REC[Q_3]$, the corresponding *Reward* value is positive. Given that the H_{decay} values are equal for both SWAP gates, the refinement function yields

$$H_{refine}(SWAP(q_2, q_3)) < H_{refine}(SWAP(q_1, q_2)), \quad (12)$$

indicating that $SWAP((q_2, q_3))$ is the optimal choice. Notably, this choice introduces a smaller circuit depth compared to $SWAP(q_1, q_2)$.

Additionally, the effectiveness of the circuit optimization strategy is demonstrated in Fig. 8. After SWAP gate decomposition and optimization, the quantum circuit in Fig. 8(c) shows a significant reduction in both the number of gates and circuit depth compared to the original circuit in Fig. 8(a). Specifically, the original circuit contains 7 CNOT gates with a depth of 6, whereas the optimized circuit includes only 5 CNOT gates and a reduced depth of 4. These results effectively demonstrate

that the proposed mapping algorithm, through the integration of SWAP adaptive decomposition and commutative gate cancellation, significantly reduces both the depth and gate count of the quantum circuit, yielding notable optimization results.

III-C Complexity Analysis

For a given quantum circuit $LC(q_l, g)$ and a quantum architecture graph $AG(Q_p, E)$, the time complexity analysis is as follows:

During the initial mapping phase, suppose there are $O(|q_l|)$ logical qubits, each of which can choose from $O(|Q_p|)$ physical qubit locations. In the worst case, the time complexity for evaluating each possible choice is $O(|q_l|)$. Since the initial mapping involves determining the positions for all logical qubits, the overall time complexity for this phase is $O(|q_l|^2 \cdot |Q_p|)$.

In the intermediate qubit routing phase, the candidate set size for each SWAP operation is $O(|E|)$, where $|E|$ represents the number of edges in the architecture graph. Assuming each evaluation enables only one gate operation, $O(|g|)$ evaluations are required, where $|g|$ is the total number of gates in the quantum circuit. Additionally, the computational complexity of the heuristic function is $O(|g|)$. Therefore, the time complexity of the routing phase is $O(|g|^2 \cdot |E|)$.

Combining the complexities of the initial mapping and intermediate routing phases, the overall time complexity of the algorithm is:

$$O(|q_l|^2 \cdot |Q_p| + |g|^2 \cdot |E|). \quad (13)$$

This complexity indicates that the algorithm's dependence on the input parameters is polynomial in nature, ensuring good scalability and computational efficiency. This is further validated by subsequent experiments, where the algorithm demonstrates reasonable runtime when handling large-scale quantum circuits.

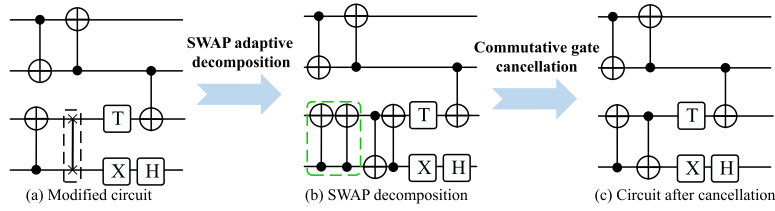


Figure 8: Quantum circuit optimized through gate cancellation and commutation

IV EXPERIMENTS AND EVALUATION

To comprehensively demonstrate the effectiveness of the quantum circuit mapping algorithm proposed in this paper, its performance was assessed across different circuit dataset sizes and architectures with varying connectivity sparsity.

IV-A Methodology

Dataset: We extracted common quantum circuits from existing benchmark tests as representative circuits for execution on the IBM Q20 architecture. These circuits consist solely of two-qubit gates (CX gates) and single-qubit gates (H, T, $T^\dagger$, etc.). Additionally, to assess the algorithm’s performance on large-scale quantum circuit sets, we selected quantum circuits used in the field of chemistry. These circuits vary in gate count, ranging from tens of thousands to hundreds of thousands, and include only two-qubit gates (CX) and some single-qubit gates (U_1 , U_3), with the number of qubits ranging from 20 to 40. These datasets provide a comprehensive evaluation of the algorithm’s performance under different scenarios.

Hardware Model: This study is primarily evaluated on two common quantum computing architectures. For circuits with fewer than 20 qubits, the IBM Q20 architecture, featuring 20 qubits and relatively dense connectivity, was chosen as the benchmark. For circuits with more qubits (ranging from 20 to 40), experiments were conducted on the Rochester architecture, which is configured with 53 qubits and relatively sparse connectivity.

Algorithm Configuration: During algorithm execution, the parameters for the initial mapping phase, α and β , were each set to 0.5, and γ was set to 0.99. Besides, δ

was set to 0.001, the positive *Reward* was set to 0.5, and the weight parameter w was also set to 0.5. Each algorithm was run three times, and the best result was selected for comparison.

Experimental Platform: All experiments were conducted on a personal computer with the following configuration: Intel(R) Core(TM) i5-8300H CPU @ 2.30GHz and 16GB DDR4 memory.

Evaluation Metrics: Under various architectures and datasets, several common metrics were compared, including the algorithm's runtime, the final number of two-qubit gates in the circuit, and the overall circuit depth. These factors are crucial for evaluating the performance of quantum algorithms.

Comparative Algorithms: We compared the proposed quantum circuit mapping algorithm with the NASSC[10] algorithm and SQGM [11] algorithm, which is designed for optimizing circuit depth. It is important to note that both NASSC and SQGM use the SABRE algorithm's initial mapping method during their initial mapping phase and have made significant contributions to the academic community.

IV-B Comparison with NASSC and

SQGM on the IBM Q20 Architecture

Table I: Comparison of CNOT count and circuit depth between NASSC and SQGM with TANGO on the IBM Q20 for 'small' circuits

name	#qubits	$CNOT_{total}$	$Depth_{total}$	NASSC			SQGM			TANGO		
				g_1	d_1	RT	g_2	d_2	RT	g_3	d_3	RT
asm	3	17	22	33	41	0.1	36	41	0.05	17	22	0.06
asm	3	23	29	39	50	0.13	43	47	0.06	23	29	0.08
8.qasm	4	23	30	27	35	0.1	30	37	0.05	21	28	0.12
qasm	4	16	21	30	38	0.1	19	25	0.04	12	19	0.04
asm	5	49	61	74	87	0.19	99	95	0.09	55	66	0.16
asm	5	38	47	54	63	0.14	63	73	0.14	44	54	0.11
l.qasm	5	16	21	22	27	0.08	29	30	0.04	16	21	0.06
asm	5	24	30	54	62	0.15	55	60	0.06	27	33	0.07
asm	5	51	66	80	89	0.25	82	87	0.1	55	70	0.11
5.qasm	5	64	84	99	115	0.28	174	163	0.17	81	98	0.43
.qasm	5	126	162	201	234	0.49	205	225	0.22	182	209	0.37
qasm	5	13	13	25	27	0.07	13	13	0.03	13	13	0.04
.qasm	5	16	21	25	31	0.09	25	28	0.04	16	21	0.05
ree-sm	5	65	82	115	137	0.3	139	137	0.14	101	114	0.21
ree-sm	5	59	76	127	136	0.32	101	103	0.1	107	111	0.26
ree-sm	5	32	40	44	53	0.16	42	50	0.06	42	53	0.11
asm	5	27	36	30	38	0.11	33	41	0.07	27	36	0.10
asm	5	36	47	57	68	0.16	83	85	0.09	44	55	0.11
mean												2

* Note: g_1, g_2, g_3 and d_1, d_2, d_3 represent the total number of CNOT gates and circuit depth after mapping with NASSC, SQGM, and TANGO algorithms,

respectively. RT refers to the runtime (s). $\Delta_{g_i} = 1 - \frac{g_3}{g_i}$, $\Delta_{d_i} = 1 - \frac{d_3}{d_i}$.



Table II: Comparison of CNOT count and circuit depth between NASSC and SQGM with TANGO on the IBM Q20 for ‘medium’ circuits

Benchmark name	#qubits	$CNOT_{total}$	$Depth_{total}$	NASSC			
				g_1	d_1	RT	g_2
4gt12-v0_87.qasm	6	112	131	218	249	0.52	263
4gt4-v0_78.qasm	6	109	137	137	158	0.32	195
4gt4-v0_79.qasm	6	105	132	175	203	0.49	189
4gt4-v0_80.qasm	6	79	101	124	144	0.29	145
4gt4-v1_74.qasm	6	119	154	211	255	0.54	277
decod24- enable_126.qasm	6	149	190	246	291	0.63	330
ex3_229.qasm	6	175	226	353	406	0.82	352
4mod5- bdd_287.qasm	7	31	41	68	75	0.2	52
C17_204.qasm	7	205	253	383	424	0.93	392
rd53_130.qasm	7	448	569	800	872	1.81	996
rd53_131.qasm	7	200	261	294	347	0.68	338
rd53_135.qasm	7	134	159	211	216	0.5	287
cm82a_208.qasm	8	283	337	520	571	1.33	501
rd53_138.qasm	8	60	56	115	118	0.3	123
urf2_152.qasm	8	35210	44100	62829	70800	146.38	75982
urf2_277.qasm	8	10066	11390	20031	20399	52.52	22940
con1_216.qasm	9	415	508	749	816	1.86	932
hwb8_113.qasm	9	30372	38717	53181	58913	140.91	67014
urf1_149.qasm	9	80878	99585	137512	154449	364.66	173067
urf1_278.qasm	9	26692	30955	48914	50696	126.27	61833
urf5_158.qasm	9	71932	89145	128754	142868	316.42	154647
urf5_280.qasm	9	23764	27822	42042	44640	119.1	53690
Geometric mean							

To comprehensively evaluate the performance of the algorithm across datasets of varying scales, circuits with five or fewer qubits are categorized as “small”, those with more than six but no more than nine qubits as “medium”, and circuits with ten or more qubits as “large”. The IBM Q20 architecture, one of the most commonly used in quantum algorithms, is widely adopted due to its dense connectivity, which introduces a more complex optimization problem. In the current dataset

of circuits, the TANGO demonstrates strong performance and is able to find near-optimal solutions. Compared to the other two heuristic-based mapping algorithms, the presented algorithm effectively reduces both gate count and circuit depth.

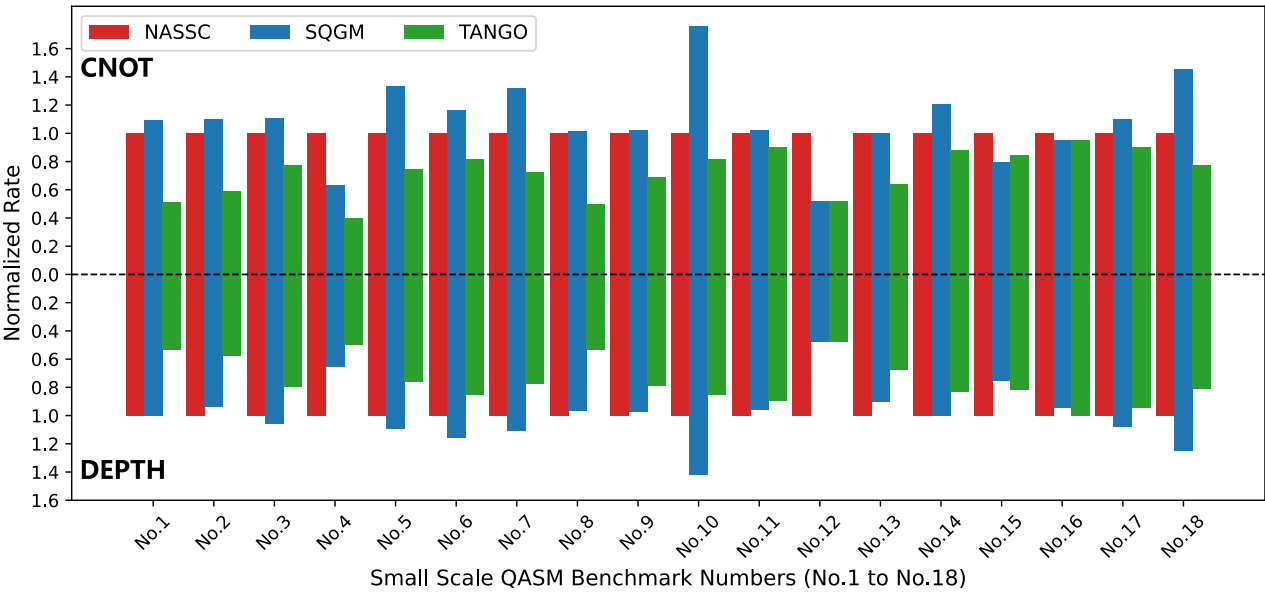


Figure 9: Comparison of total CNOT gates and circuit depth for small-scale quantum circuits on IBM Q20 (normalize to NASSC)

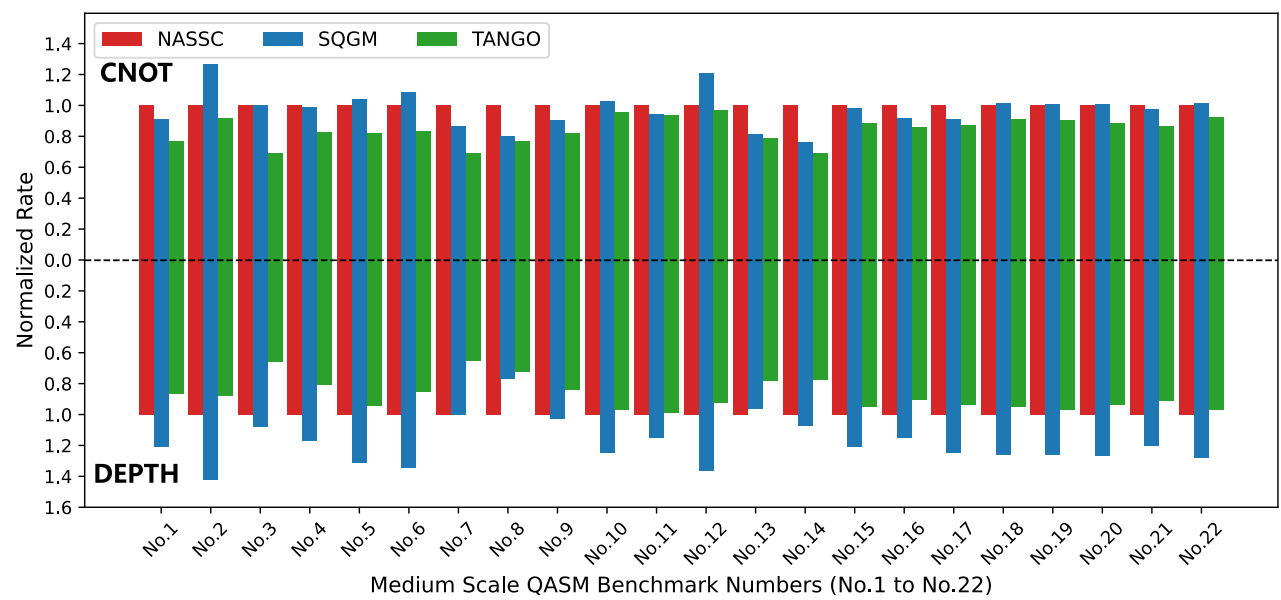


Figure 10: Comparison of total CNOT gates and circuit depth for medium-scale quantum circuits on IBM Q20 (normalize to NASSC)

Table III: Comparison of CNOT count and circuit depth between NASSC and SQGM with TANGO on the IBM Q20 for 'large' circuits

Benchmark name	#qubits	$CNOT_{total}$	$Depth_{total}$	NASSC		
				g_1	d_1	RT
hwb9_119.qasm	10	90955	116199	169675	184589	437.96
mini_alu_305.qasm	10	77	69	114	100	0.28
qft_10.qasm	10	90	63	49	38	0.31
urf3_155.qasm	10	185276	229365	338670	371971	864.05
urf3_279.qasm	10	60380	70702	115498	118828	324.6
9symml_195.qasm	11	15232	19235	27076	29351	69.52
dc1_220.qasm	11	833	1038	1481	1622	3.6
life_238.qasm	11	9800	12511	17351	18902	42.87
urf4_187.qasm	11	224028	264330	378188	398902	1000.23
wim_266.qasm	11	427	514	717	800	1.63
z4_268.qasm	11	1343	1644	2457	2648	5.72
cycle10_2_110.qasm	12	2648	3386	4791	5285	12.28
sqr8_260.qasm	12	1314	1659	2335	2495	11.58
sym9_146.qasm	12	148	127	262	219	0.63
plus63mod4096_163.qasm	13	56329	72246	101544	110144	268.3
radd_250.qasm	13	1405	1781	2707	2955	6.13
rd53_311.qasm	13	124	124	256	283	0.62
0410184_169.qasm	14	104	104	148	144	0.38
cm42a_207.qasm	14	771	940	1323	1421	3.04
plus63mod8192_164.qasm	14	81865	105142	145446	158513	355.55
pm1_249.qasm	14	771	940	1348	1522	3.16
sao2_257.qasm	14	16864	19563	28867	30118	72.63
ham15_107.qasm	15	3858	4819	6636	7338	17.17
misex1_241.qasm	15	2100	2676	3343	3708	11.24
cnt3-5_180.qasm	16	215	209	363	367	0.98
inc_237.qasm	16	4636	5863	7869	8606	24.78
mlp4_245.qasm	16	8232	10328	15421	16547	37.96
Geometric mean						

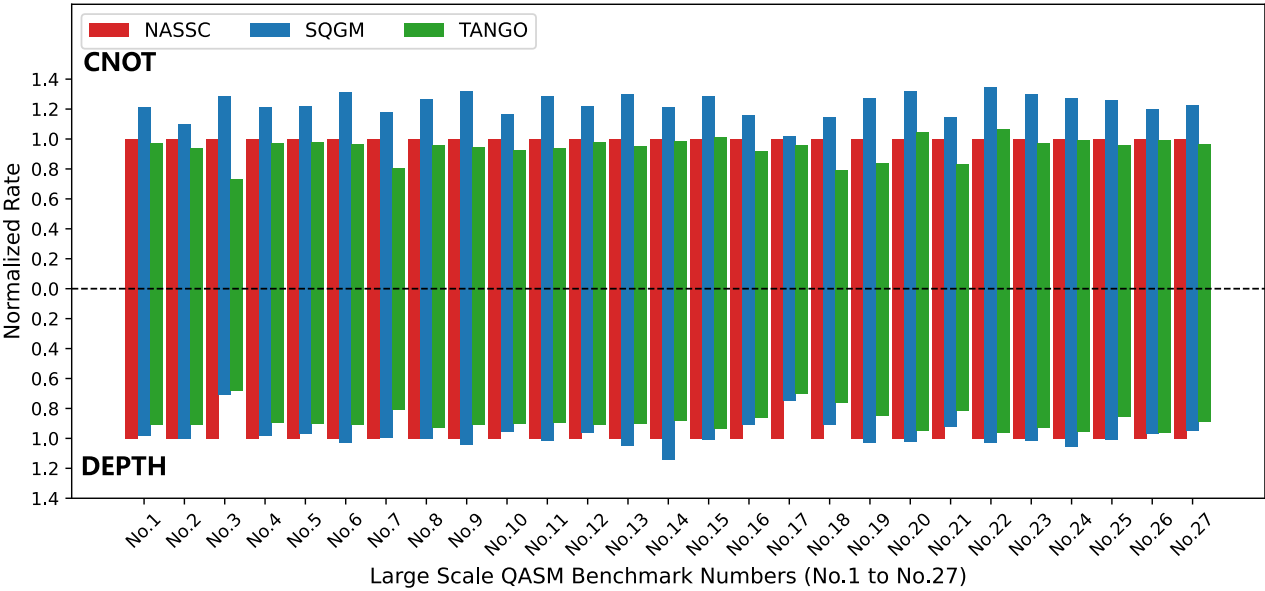


Figure 11: Comparison of total CNOT gates and circuit depth for large-scale quantum circuits on IBM Q20 (normalize to NASSC)

The experimental results for small-scale quantum circuits are presented in Table. I and Fig. 9. The table provides detailed information on each quantum circuit, including the number of qubits, CNOT gate count, and circuit depth. It also reports the two primary objectives, CNOT gate count and circuit depth, along with the algorithm runtime (in seconds) after applying different mapping algorithms. Additionally, the table highlights the optimization rates of the proposed algorithm in terms of CNOT gate reduction and circuit depth improvement compared to the other two algorithms. Specifically, TANGO achieves a significant reduction in both objectives compared to the NASSC algorithm. For instance, in the circuit *rd32-v1_68*, the gate count is reduced by up to 60%, and on average, it reduces the gate count by 27.85% across all small circuits. In terms of circuit depth, Our methodology reduces depth by 50% in the best case, with an average reduction of 25.35%, indicating notable progress in optimization.

Furthermore, it is worth noting that in most cases, the runtime of the TANGO algorithm is shorter than that of the NASSC algorithm. This is primarily due to the fact that the NASSC algorithm requires the re-synthesis of gate blocks, which incurs high computation. In comparison with the SQGM algorithm, the proposed algo-

rithm also shows strong optimization in both circuit depth and overall gate count. Since the SQGM algorithm primarily focuses on optimizing circuit depth, we observe that its gate count optimization lags behind that of NASSC, despite its superior performance in reducing circuit depth. Specifically, in terms of gate count optimization, our algorithm reduces the gate count by up to 53.45%, particularly in the circuit *decod24-v3_45*, and on average, it reduces the gate count by 30.92% across the entire small dataset. For circuit depth optimization, TANGO achieves a reduction of 46.34% in the best case, particularly in the circuit *3_17_13*, and an average reduction of 22.59% across all small circuits.

However, in terms of algorithm runtime, the SQGM algorithm outperforms our TANGO algorithm. This is mainly due to the staged selection strategy in the presented algorithm's routing phase, which selects the most suitable SWAP gates, thereby increasing the computational overhead. Therefore, although the runtime of the algorithm is longer than that of SQGM on several quantum circuits, it remains within an acceptable range.

Similarly, good optimization results were also observed in the medium-scale quantum circuits, as shown in the Table. II and Fig. 10, where a certain level of optimization was consistently achieved. Among these circuits, the most significant optimization was seen in the *4gt4-v0_79* quantum circuit, where TANGO outperformed both the NASSC and SQGM algorithms. In this circuit, it achieved an optimization rate exceeding 30% in both gate count and circuit depth. However, it is noteworthy that the overall optimization performance of the devised algorithm on medium-scale quantum circuits is lower than that on small quantum circuits. Specifically, in terms of gate count and circuit depth optimization, TANGO improved by 12.92% and 15.38%, respectively, compared to the NASSC algorithm. When compared to the SQGM algorithm, our algorithm achieved improvements of 25.69% and 12.76% in gate count and circuit depth, respectively.

Table IV: Comparison of CNOT count and circuit depth between NASSC and SQGM with TANGO on the IBM Rochester for chemistry-related circuits

Benchmark name	#qubits	$CNOT_{total}$	$Depth_{total}$	NASSC		
				g_1	d_1	RT
H2_cmplt_JW_ccpvdz.qasm	20	14616	16435	15934	16947	33.4
NH_frz_JW_631g.qasm	20	88536	95290	125540	126957	372.1
LiH_cmplt_JW_631g.qasm	22	69144	74422	77302	79449	238.9
NH_cmplt_JW_631g.qasm	22	173264	183892	271456	271242	723.8
C2H4_cmplt_JW_sto3g.qasm	24	640768	662500	1061760	1055078	10596
H2O_cmplt_JW_631g.qasm	24	414240	431664	627136	631831	2632.
H2O_frz_JW_631g.qasm	26	244992	257780	292766	285822	3470.
C2H4_frz_JW_sto3g.qasm	28	312096	327532	563229	548683	2051.
LiH_frz_JW_ccpvdz.qasm	36	89080	95507	194069	193187	392.6
LiH_frz_P_ccpvdz.qasm	36	86598	93681	138260	135697	364.8
LiH_cmplt_JW_ccpvdz.qasm	38	407320	419830	607302	580325	2236.
H4_cmplt_JW_ccpvdz.qasm	40	479136	492286	738062	721920	9527
H4_cmplt_P_ccpvdz.qasm	40	472132	510907	777749	757353	3081.
Geometric mean						

For large-scale quantum circuits in Table. III and Fig. 11, we used the ultra-large-scale circuit *urf4_187*, which contains 224,028 two-qubit gates and has a circuit depth of 264,330, as an example. Compared to the NASSC algorithm, the TANGO algorithm achieved a 5.32% improvement in gate count and an 8.85% improvement in circuit depth. As previously mentioned, the SQGM algorithm primarily focuses on optimizing circuit depth, resulting in weaker performance in gate count optimization. In contrast, our approach demonstrated a significant improvement of 28.11% in gate count compared to the SQGM algorithm. However, it is also observed that the optimization rate of the TANGO algorithm on large-scale quantum circuits is lower than on smaller ones, which may indicate that the heuristic algorithm is more likely to get trapped in local optima when searching within a larger solution space. Overall, compared to the NASSC algorithm, the presented algorithm achieved an average optimization rate of 5.95% in gate count and an 11.88% improvement in circuit depth. When compared to the SQGM algorithm, it performed

excellently in gate count optimization, with a 23.35% improvement, but showed a slight decrease in circuit depth optimization, achieving a 9.74% improvement.

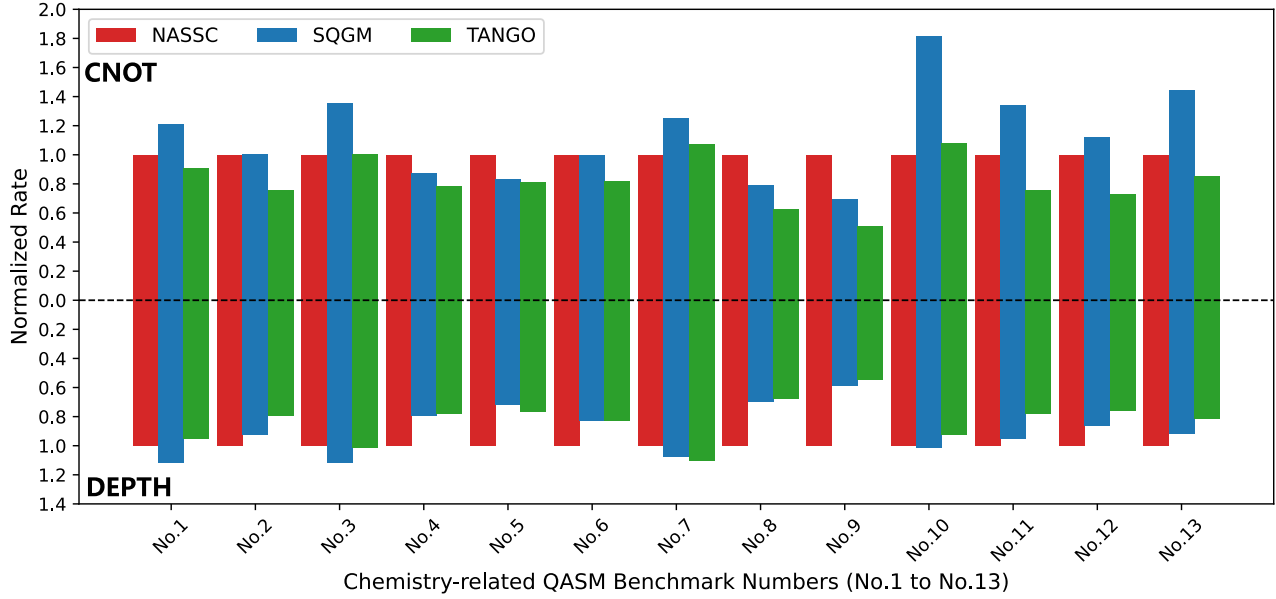


Figure 12: Comparison of total CNOT gates and circuit depth for chemistry-related quantum circuits on IBM Rochester Q53 (normalize to NASSC)

IV-C Comparison with NASSC and SQGM on the Rochester Architecture

To validate the performance of the algorithm in high-qubit quantum circuits, a set of quantum chemistry-related circuits was selected. These circuits are characterized by a large number of qubits and a high density of quantum gates, making them ideal for testing the algorithm's performance in complex scenarios. For the experiments, the Rochester architecture was chosen for the qubit coupling structure in the experiments. This architecture features relatively sparse connectivity, which typically supports the expansion to a larger number of qubits. As connectivity increases, the interference between qubits also intensifies, making this sparse-connectivity architecture a focal point for researchers.

The experimental results presented in Table. IV and Fig. 12 show that the TANGO algorithm outperforms the NASSC algorithm in 10 out of 13 quantum circuits, demonstrating effective optimization of both gate count and circuit depth. The maximum gate count optimization rate achieved was 48.80%, observed in the *LiH_frz_JW_ccpvdz* circuit, with an average optimization rate of 17.61% across this dataset. Regarding depth optimization, the proposed approach achieved an average optimization rate of 17.43%. In terms of overall runtime, it outperformed the NASSC algorithm with shorter runtime. However, in the *LiH_cmplt_JW_631g* and *H2O_frz_JW_631g* circuits, the devised algorithm showed negative optimization in both gate count and circuit depth, which may be due to the structure of these circuits being more suited to the block re-synthesis module of the NASSC algorithm.

When compared to the SQGM algorithm, the TANGO algorithm consistently outperforms SQGM in gate count optimization across all quantum circuits, achieving an average optimization rate of 25.19%. However, in terms of circuit depth optimization, our designed algorithm's performance is less impressive, with a maximum optimization rate of 18.48% and an average rate of 7.03%. This result indirectly suggests that reducing the number of gates in a circuit does not always lead to a significant reduction in its depth. Therefore, finding a balance between gate count and circuit depth, such that both optimization objectives fall within an acceptable range, is the key challenge addressed in this paper.

V CONCLUSION

In this paper, we propose a quantum circuit mapping algorithm called TANGO, which jointly optimizes both circuit depth and gate count. Specifically, during the initial mapping phase, we design a dual-factor initial mapping algorithm that considers the influence between mapped and unmapped qubits and defines relevant parameters to balance these two components, thus improving the performance of initial mapping. In the qubit routing phase, we propose a two-stage qubit routing algorithm. Using the number of executable gates as the primary evaluation metric, and combining quantum gate distance, circuit depth, and bidirectional-look SWAP strategy, the method effectively identifies the SWAP gates with optimal exchange efficiency and minimal circuit depth. Finally, by integrating advanced quantum gate optimization techniques, the total gate count and depth of the circuit are further reduced. Compared to existing state-of-the-art algorithms, our algorithm

achieves significant optimizations in both gate count and circuit depth across different datasets and architectures, with no substantial increase in runtime. These results demonstrate the scalability and adaptability of our algorithm, offering a viable solution to the quantum mapping problem.

acknowledgments

This research was supported by the National Nature Science Foundation of China (Grant No. 62101600, Grant No. 62471070), and the State Key Lab of Processors, Institute of Computing Technology, CAS (Grant No. CLQ202404).

References

- [1] J. Chow, O. Dial, and J. Gambetta, “Ibm quantum breaks the 100-qubit processor barrier,” *IBM Research Blog*, 2021.
- [2] F. Arute, K. Arya, R. Babbush, D. Bacon, J. C. Bardin, R. Barends, R. Biswas, S. Boixo, F. G. Brandao, D. A. Buell *et al.*, “Quantum supremacy using a programmable superconducting processor,” *Nature*, vol. 574, no. 7779, pp. 505–510, 2019.
- [3] J. Hsu, “Ces 2018: Intel’s 49-qubit chip shoots for quantum supremacy,” *IEEE Spectrum Tech Talk*, pp. 1–6, 2018.
- [4] G. Q. Ai and Collaborators, “Quantum error correction below the surface code threshold,” *arXiv preprint arXiv:2408.13687*, 2024.
- [5] A. Zulehner, A. Paller, and R. Wille, “An efficient methodology for mapping quantum circuits to the ibm qx architectures,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 38, no. 7, pp. 1226–1236, 2018.
- [6] G. Li, Y. Ding, and Y. Xie, “Tackling the qubit mapping problem for nisq-era quantum devices,” in *Proceedings of the twenty-fourth international conference on architectural support for programming languages and operating systems*, 2019, pp. 1001–1014.

- [7] S. Niu, A. Suau, G. Staffelbach, and A. Todri-Sanial, “A hardware-aware heuristic for the qubit mapping problem in the nisq era,” *IEEE Transactions on Quantum Engineering*, vol. 1, pp. 1–14, 2020.
- [8] C. Zhang, A. B. Hayes, L. Qiu, Y. Jin, Y. Chen, and E. Z. Zhang, “Time-optimal qubit mapping,” in *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2021, pp. 360–374.
- [9] K.-Y. Chang and C.-Y. Lee, “Mapping nearest neighbor compliant quantum circuits onto a 2-d hexagonal architecture,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 41, no. 10, pp. 3373–3386, 2021.
- [10] J. Liu, P. Li, and H. Zhou, “Not all swaps have the same cost: A case for optimization-aware qubit routing,” in *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2022, pp. 709–725.
- [11] S. Li, K. D. Nguyen, Z. Clare, and Y. Feng, “Single-qubit gates matter for optimising quantum circuit depth in qubit mapping,” in *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. IEEE, 2023, pp. 1–9.
- [12] E. McKinney, M. Hatridge, and A. K. Jones, “Mirage: Quantum circuit decomposition and routing collaborative design using mirror gates,” in *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2024, pp. 704–718.
- [13] D. B. Tan, W.-H. Lin, and J. Cong, “Compilation for dynamically field-programmable qubit arrays with efficient and provably near-optimal scheduling,” *arXiv preprint arXiv:2405.15095*, 2024.
- [14] R. Wille, O. Keszocze, M. Walter, P. Rohrs, A. Chattopadhyay, and R. Drechsler, “Look-ahead schemes for nearest neighbor optimization of 1d and 2d quantum circuits,” in *2016 21st Asia and South Pacific design automation conference (ASP-DAC)*. IEEE, 2016, pp. 292–297.
- [15] X. Zhou, S. Li, and Y. Feng, “Quantum circuit transformation based on simulated annealing and heuristic search,” *IEEE Transactions on*

- [16] S. Li, X. Zhou, and Y. Feng, “Qubit mapping based on subgraph isomorphism and filtered depth-limited search,” *IEEE Transactions on Computers*, vol. 70, no. 11, pp. 1777–1788, 2020.
- [17] K. Ye, S. Dai, B. Guo, Y. Shen, C. Liu, K. Bi, F. Chen, Y. Hu, and M. Zhao, “A mutual-influence-aware heuristic method for quantum circuit mapping,” *IEEE Transactions on Computers*, 2024.
- [18] J. Liu, E. Younis, M. Weiden, P. Hovland, J. Kubiawicz, and C. Iancu, “Tackling the qubit mapping problem with permutation-aware synthesis,” in *2023 IEEE International Conference on Quantum Computing and Engineering (QCE)*, vol. 1. IEEE, 2023, pp. 745–756.
- [19] H. Jiang, Y.-X. Deng, and M. Xu, “Qubit mapping based on tabu search,” *Journal of Computer Science and Technology*, vol. 39, no. 2, pp. 421–433, 2024.
- [20] Y. Qian, Z. Guan, S. Zheng, and S. Feng, “A method based on timing weight priority and distance optimization for quantum circuit transformation,” *Entropy*, vol. 25, no. 3, p. 465, 2023.
- [21] S. Park, D. Kim, M. Kweon, J.-Y. Sim, and S. Kang, “A fast and scalable qubit-mapping method for noisy intermediate-scale quantum computers,” in *Proceedings of the 59th ACM/IEEE Design Automation Conference*, 2022, pp. 13–18.
- [22] P. Zhu, S. Feng, and Z. Guan, “An iterated local search methodology for the qubit mapping problem,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 41, no. 8, pp. 2587–2597, 2021.
- [23] C.-Y. Huang and W.-K. Mak, “Efficient qubit routing using a dynamically-extract-and-route framework,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2024.
- [24] L. Lao, H. Van Someren, I. Ashraf, and C. G. Almudever, “Timing and resource-aware mapping of quantum circuits to superconducting processors,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 41, no. 2, pp. 359–371, 2021.

- [25] Y. Li, Y. Zhang, M. Chen, X. Li, and P. Xu, “Timing-aware qubit mapping and gate scheduling adapted to neutral atom quantum computing,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 42, no. 11, pp. 3768–3780, 2023.
- [26] P. Escofet, A. Gonzalvo, E. Alarcón, C. G. Almudéver, and S. Abadal, “Route-forcing: Scalable quantum circuit mapping for scalable quantum computing architectures,” in *2024 IEEE International Conference on Quantum Computing and Engineering (QCE)*, vol. 1. IEEE, 2024, pp. 909–920.
- [27] H. Fu, M. Zhu, J. Wu, W. Xie, Z. Su, and X.-Y. Li, “Effective and efficient qubit mapper,” in *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. IEEE, 2023, pp. 1–9.
- [28] P. Murali, J. M. Baker, A. Javadi-Abhari, F. T. Chong, and M. Martonosi, “Noise-adaptive compiler mappings for noisy intermediate-scale quantum computers,” in *Proceedings of the twenty-fourth international conference on architectural support for programming languages and operating systems*, 2019, pp. 1015–1029.
- [29] S. S. Tannu and M. K. Qureshi, “Not all qubits are created equal: A case for variability-aware policies for niscq-era quantum computers,” in *Proceedings of the twenty-fourth international conference on architectural support for programming languages and operating systems*, 2019, pp. 987–999.
- [30] H. Deng, Y. Zhang, and Q. Li, “Codar: A contextual duration-aware qubit mapping for various niscq devices,” in *2020 57th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2020, pp. 1–6.
- [31] P. Zhu, W. Ding, L. Wei, X. Cheng, Z. Guan, and S. Feng, “A variation-aware quantum circuit mapping approach based on multi-agent cooperation,” *IEEE Transactions on Computers*, vol. 72, no. 8, pp. 2237–2249, 2023.
- [32] C.-Y. Huang and W.-K. Mak, “Ctqr: Control and timing-aware qubit routing,” in *2024 29th Asia and South Pacific Design Automation Conference (ASP-DAC)*. IEEE, 2024, pp. 140–145.

[33] A. Ash-Saki, M. Alam, and S. Ghosh, “Qure: Qubit re-allocation in noisy intermediate-scale quantum computers,” in *Proceedings of the 56th Annual Design Automation Conference 2019*, 2019, pp. 1–6.



Feeling
lucky?

Conversion
report (W)

Report
an issue (<https://github.com/dginev/ar5iv/issues/new?template=improve-article--arxiv-id-.md&title=Improve+article+2503.07331>)

View original
on arXiv (<https://arxiv.org/abs/2503.07331>)



Copyright (<https://arxiv.org/help/license>)

Privacy Policy (https://arxiv.org/help/policies/privacy_policy)

Generated by  (LaTeX) (<http://dmlf.nist.gov/LaTeXML/>) oxide